Student Name: Pham Xuan Hoang
Student ID: 2640009

Information Systems
Labwork 5

`employee` table

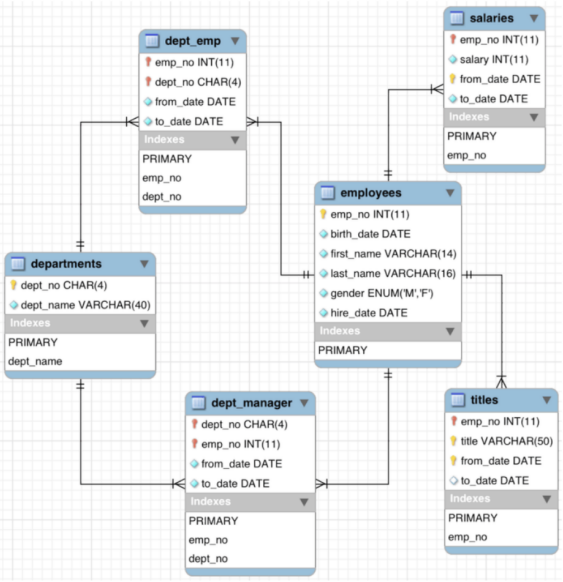


Figure 1: `employees` database schema

1NF. All column holds a single atomic value, there is no column that has a list of values and no table contains repeated columns. Every table has a primary key (some has a composite primary key) so there are no identical rows.

2NF. We can remove all the tables from the analysis that doesn't have a composite primary key first. They are automatically in 2NF form.

- `employees` only has 1 primary key `emp_no`
- `departments` only has 1 primary key `departments`

Now we can analyze the tables with composite primary key.

- `dept_emp` and `dept_manager` tables each have one composite primary key, both of them are also foreign key: `emp_no` (links to `employees`) and `dept_no` (links to `departments`). `from_date` and `to_date` need the full composite foreign key to be identifiable
  - Remove `emp_no`: the time frame collapses into each department, we can no longer distinguish the time frame of employee within a department.
  - Remove `dept_no`: the time frame only links with employees. If an employee changed department, we cannot add a new row as the PK has already been populated.

- `titles` tables has a composite key composed of `emp_no`, `dept_no`, of which `emp_no` is the foreign key linking to `employees`. The non-key attribtue `to_date` requires all of them to be distinct.
  - An employee can have multiple titles.
  - One title can be held by many employees.
  - An employee can leave a title then return, which is why we need `from_date`.
- `salaries` has a composite primary key of `emp_no` and `from_date`. Again, the two non-key columns `salary` and `to_date` depend on the entire composite key to be distinct.
  - Remove `emp_no`: no way to tell which employee get a raise.
  - Remove `from_date`: multiple employee can get a new salary at the same time in different amount.

3NF. No non-key column determines another non-key column:

- `employees`: none of determines another.
- `departments`: the only non-key column, `dept_name`, is `UNIQUE`, so it is a second candidate key, not a transitive dependency.
- `dept_emp`, `dept_manager`: between `from_date` and `to_date`, neither determines the other.
- `titles`: single non-key column (`to_date`), so no transitive dependency.
- `salaries`: `salary` has no connection to `to_date` and vice versa.

All of the points above can be summarized into the table below

Table	**PK**	**1NF: Columns with atomic value, no list in column**	**2NF: Is there any composite key?**	**3NF: Are there any non-key → non-key?**
`employees`	`emp_no`	`birth_date`, `first_name`, `last_name`, `gender`, `hire_date`	None, only `emp_no`	None, all attributes depend on `emp_no`
`departments`	`dept_no`	`dept_name`	None, only `dept_no`	None, only one attribute and one primary key
`dept_emp`	`emp_no` `dept_no`	`from_date`, `to_date`	1 composite PK `dept_no` and `emp_no`, both are primary keys and both are foreign keys too	None, `from_date` and `to_date` are independent
`dept_manager`	`emp_no` `dept_no`	`from_date`, `to_date`	Same as `dept_emp`	Same as `dept_emp`
`titles`	`emp_no` `title` `from_date`	`to_date`	1 composite PK `emp_no`, `title` and `from_date` with `emp_no` is also a foreign key	None, `to_date` depends on the entire composite key
`salaries`	`emp_no` `from_date`	`salary`, `to_date`	1 composite key of `emp_no` and `from_date` with `emp_no` is also a foreign key	None, `salary` and `to_date` requires the entire composite key

Table 1: Normalization summary of the `employees` database

Order form

Normalize the following document to 3NF

Order Form			
Order number	on7126		
Order date	11/07/2016		
Customer name	Tran Giang Son		
Customer phone	+84-4 37 91 69 60		
Customer address	18 Hoang Quoc Viet		
City	Hanoi, Vietnam		
ProdId	Description	Quantity	Price
P101	Intel i7 6700K	1	$349
P102	ASUS Z170A-P	1	$149
P103	Corsair DDR4 4 × 8GB	1	$469
P104	Samsung EVO 850 512GB	1	$249
P105	GeForce GTX 1080 8G	1	$725
P106	Corsair CS750M 80+ Gold	1	$119
P107	Corsair Carbide 400R Mid	1	$85

Table 2: Order form example

Step 1: Raw requirements from the order form:

```
Order(orderNo, date, customerName, customerPhone, customerAddress, city, {prodId,
description, quantity, price})
```

So we can deduce the concepts that need to be stored:

- Order
- Customer
- Product

Step 2: Determine attributes of each concepts

- Customer
  - `customerId`
  - `name`
  - `phone`
  - `address`
  - `city`
- Order
  - `orderNo`
  - `date`
- Product
  - `productId`
  - `description`
  - `price`
  - `quantity`

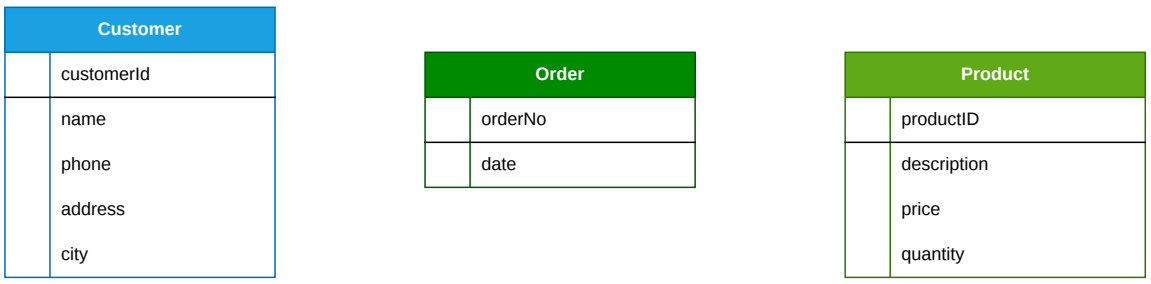


Figure 2: Order database concepts with attributes

Step 3: Determine links between the concepts

- Order and Product (has, many-to-many)
- Customer and Order (has, one-to-many)

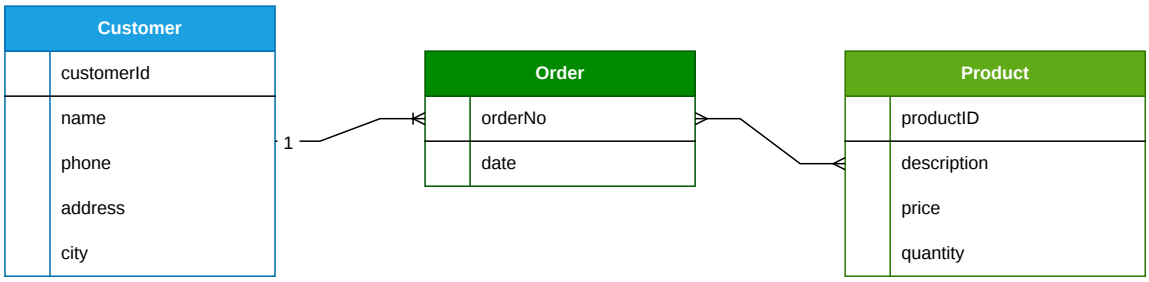


Figure 3: Order's links between concepts

Step 4: Determine types of each concept attribute

- Customer
  - `customerId: INT`
  - `name: VARCHAR(50)`
  - `phone: VARCHAR(50)`
  - `address: VARCHAR(100)`
  - `city: VARCHAR(50)`
- Order
  - `orderNo: VARCHAR(50)`
  - `date: DATE`
- Product
  - `productId: VARCHAR(50)`
  - `description: VARCHAR(100)`
  - `price: INT`
  - `quantity: INT`

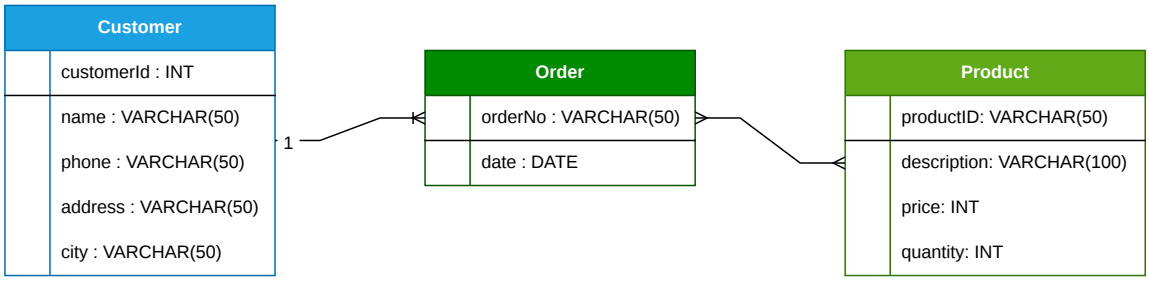


Figure 4: Order's concepts with links and attributes

Step 5: Solve foreign key links
Add primary key, add foreign key for one-to-many links and add table for many-to-many links

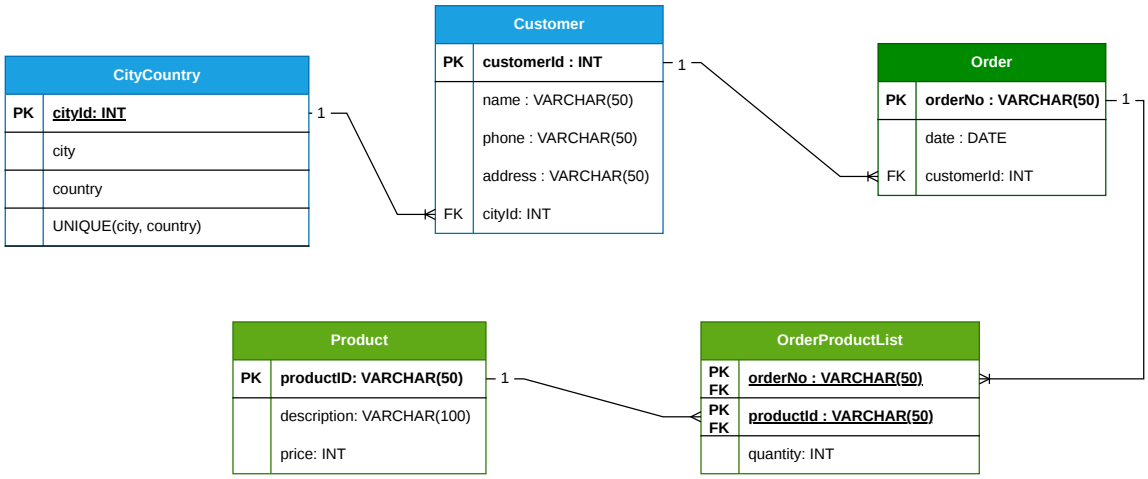


Figure 5: Order database schema normalized to 3NF

Labwork 1

You need a database to manage information about the magazines that you buy habitually.

For each magazine, its title, the ISSN (code that identifies the publication), number and year are stored. You also need data about the articles, including its title, page of beginning and page of ending. It is assumed that there are no two articles with the same title.

Every article can be written by several authors, whose name, email address and ascription are stored. Besides, a number that indicates the order of has to be stored.

Step 1: Raw requirements:

```
Magazine(
  title, ISSN, number, year,
  Articles{
    title(distinct), pageOfBeginning, pageOfEnding,
    Author{
      name, email, ascription, orderNo
    }
  }
)
```

We can deduce the concepts that need to be stored:

- Magazine
- Article
- Author

Step 2: Determine attributes of each concepts

- Magazine
  - `magazineId`
  - `title`
  - `ISSN`
  - `number`
  - `year`
- Article
  - `articleId`
  - `title`
  - `pageOfBeginning`
  - `pageOfEnding`
- Author
  - `authorId`
  - `name`
  - `email`
  - `ascription`

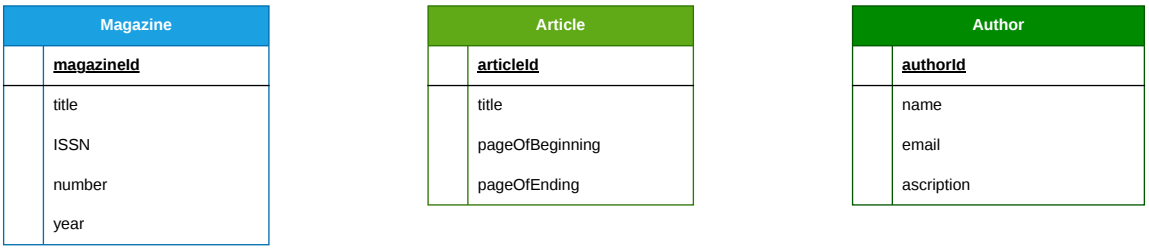


Figure 6: Magazine database concepts with attributes

`orderNo` is not an attribute of `Author`: it gives the position of an author *within one article*, so it describes the link between the two.

Step 3: Determine links between the concepts

- Magazine and Article (has, one-to-many)
- Article and Author (written by, many-to-many, the link carries `orderNo`)

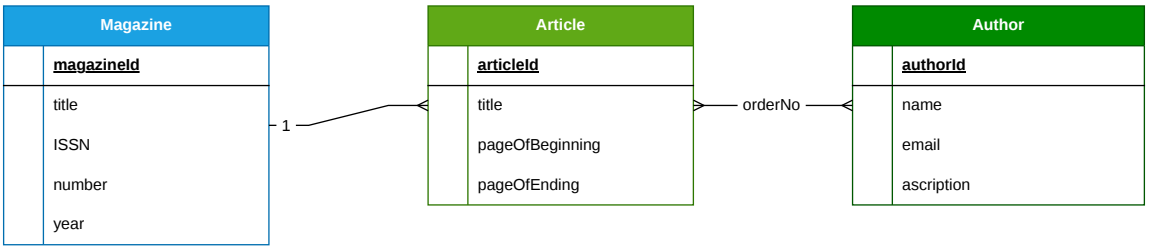


Figure 7: Magazine's links between concepts

Step 4: Determine types of each concept attribute

- Magazine
  - `magazineId`: `VARCHAR(50)`
  - `title`: `VARCHAR(100)`
  - `ISSN`: `CHAR(8)`
  - `number`: `INT`
  - `year`: `INT`
- Article
  - `articleId`: `VARCHAR(50)`
  - `title`: `VARCHAR(200)`
  - `pageOfBeginning`: `INT`
  - `pageOfEnding`: `INT`
- Author
  - `authorId`: `VARCHAR(50)`
  - `name`: `VARCHAR(50)`
  - `email`: `VARCHAR(50)`
  - `ascription`: `VARCHAR(100)`

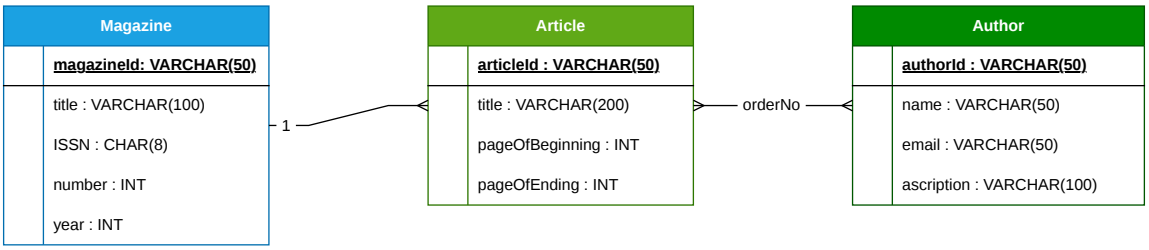


Figure 8: Magazine's concepts with links and attribute types

Step 5: Solve foreign key links
Add primary key, add foreign key for one-to-many links and add table for many-to-many links

- The many-to-many link becomes the table `ArticleAuthor`, whose composite primary key is the pair of foreign keys `articleId`, `authorId`.
- `orderNo` moves into `ArticleAuthor`, because it depends on the whole (article, author) pair.
- `Article.title` is `UNIQUE`.

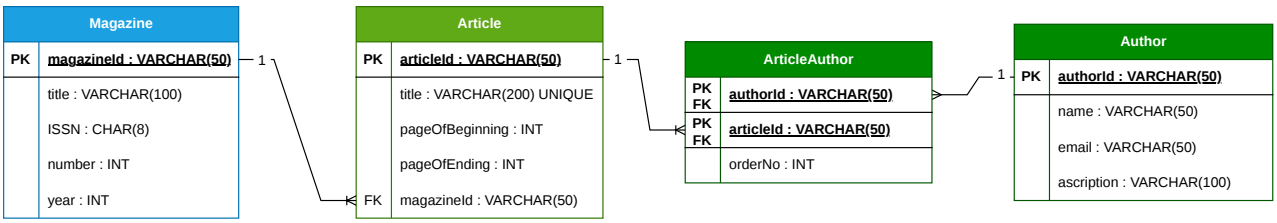


Figure 9: Magazine database schema

Labwork 2

A Non-Government Organization (NGO) wants to prepare a database for all its projects. It has diverse head offices in several countries which manage and coordinate the projects of that country.

For head offices, an identifier, the city and country where it is located, its address, a phone number and the name of the director are stored. Every head office manages a set of projects, with a code, a title, dates of beginning and end, the assigned budget and the name of the person in charge. One project is formed by a set of actions that can affect to several cities.

We want to know what actions are realised in which city, storing its name, country and number of inhabitants. We also need an identifier to differentiate them. Also there is a need of investment of the project that corresponds to the city and a small description of the action.

Step 1: Raw requirements

```
HeadOffice(
    identifier, city, country, address, phoneNumber, directorName,
    Project(
        code, title, beginningDate, endDate, budget, personInChargeName,
        Action(
            description,
            City(
                identifier, name, country, numberOfInhabitants,
                investment
            )
        )
    )
)
```

We can deduce the concepts that need to be stored:

- HeadOffice
- Project
- Action
- City

The `investment` is not a concept of its own: it is the amount invested by an action *in one city*, so it describes the link between Action and City.

Step 2: Determine attributes of each concepts

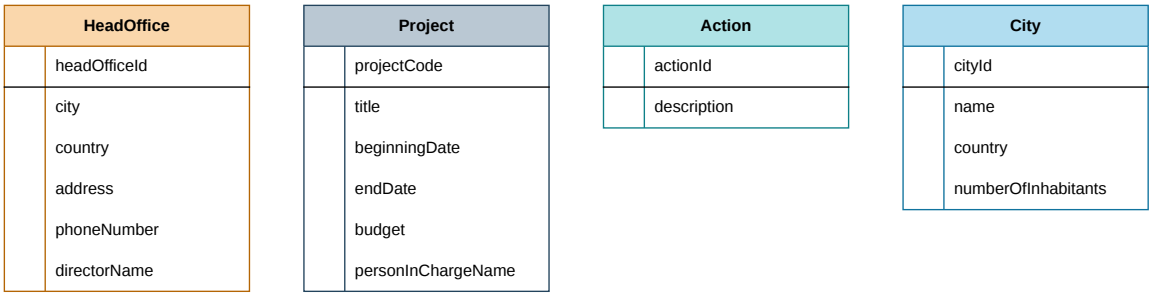


Figure 10: NGO database concepts with attributes

- HeadOffice
  - `headOfficeId`
  - `city`
  - `country`
  - `address`
  - `phoneNumber`
  - `directorName`
- Action
  - `actionId`
  - `description`
- Project
  - `projectCode`
  - `title`
  - `beginningDate`
  - `endDate`
  - `budget`
  - `personInChargeName`
- City
  - `cityId`
  - `name`
  - `country`
  - `numberOfInhabitants`

Step 3: Determine links between the concepts

- HeadOffice and Project (manages, one-to-many)
- Project and Action (formed by, one-to-many)
- Action and City (affects, many-to-many, the link carries `investment`)

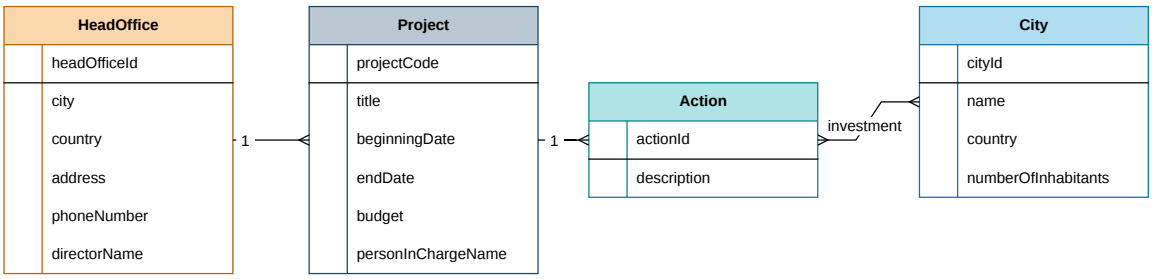


Figure 11: NGO's links between concepts

Step 4: Determine types of each concept attribute

- HeadOffice
  - `headOfficeId`: `VARCHAR(50)`
  - `city`: `VARCHAR(50)`
  - `country`: `VARCHAR(50)`
  - `address`: `VARCHAR(100)`
  - `phoneNumber`: `VARCHAR(20)`
  - `directorName`: `VARCHAR(50)`
- Action
  - `actionId`: `VARCHAR(50)`
  - `description`: `VARCHAR(255)`
- Project
  - `projectCode`: `VARCHAR(50)`
  - `title`: `VARCHAR(100)`
  - `beginningDate`: `DATE`
  - `endDate`: `DATE`
  - `budget`: `DECIMAL(15,2)`
  - `personInChargeName`: `VARCHAR(50)`
- City
  - `cityId`: `VARCHAR(50)`
  - `name`: `VARCHAR(50)`
  - `country`: `VARCHAR(50)`
  - `numberOfInhabitants`: `INT`

Action many-to-many City - `investment`: `DECIMAL(15,2)`.

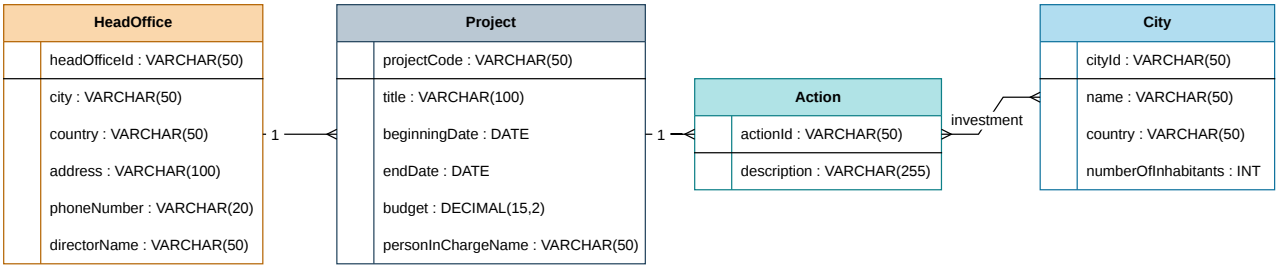


Figure 12: NGO concepts with links and attribute types

Step 5: Solve foreign key links Add primary key, add foreign key for one-to-many links and add table for many-to-many links

- The many-to-many link becomes the `ActionCity` table, whose composite primary key is the pair of foreign keys `actionId` and `cityId`.
- `investment` moves into `ActionCity`

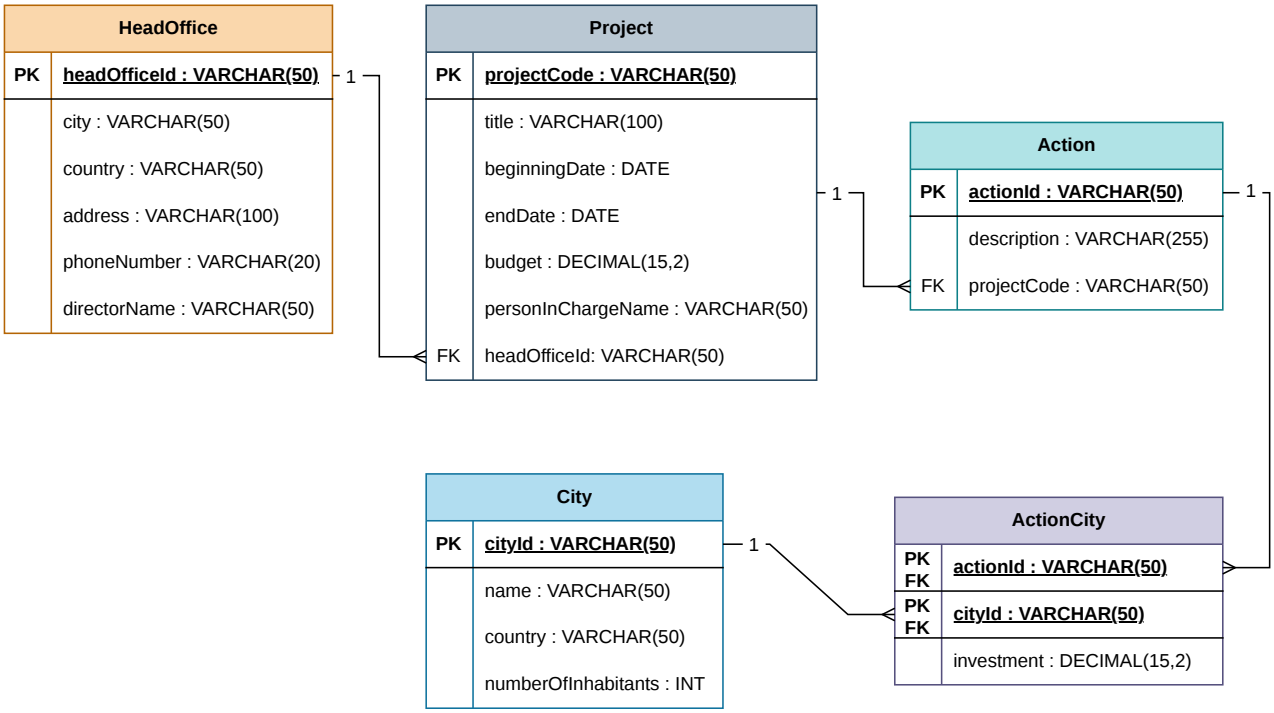


Figure 13: NGO database schema